PerturbDiff 从 Methods 到 Code 教学文档

这份文档面向 Python 初学者，目标不是只告诉你“代码怎么写”，而是帮你把三件事连起来：

1. 这个项目想解决什么问题
2. 为什么代码要这样组织
3. 如果不用当前写法，还有哪些常见替代方案

如果你一边看这份文档，一边顺着源码里的注释读代码，应该能把整个项目的主流程串起来。

1. 先用一句话理解整个项目

这个项目实现的是一个**条件扩散模型**，用来预测单细胞在某种 perturbation（比如某个 target gene 或某条 sgRNA）下的表达响应。

它的核心思想可以概括成：

```
先把细胞表达编码成 latent 表示
再把 perturbation 表示成 latent 空间里的一个“偏移向量”
最后让扩散模型根据这个条件去重建/生成目标表达
```

README 里给出的最关键公式是：

```
z_condition = E(x_control) + delta_g
x_pred = DDIM(z_condition)
```

你可以把它当作读整个仓库时的“总导航图”。

2. 推荐的读代码顺序

对于初学者，最不容易迷路的顺序不是“哪份代码最底层就先读哪份”，而是先看主流程，再补细节。

推荐顺序：

1. `scripts/train_with_split.py`
2. `src/perturbdiff/data.py`
3. `src/perturbdiff/config.py`

4. `src/perturbdiff/model.py`
5. `src/perturbdiff/modules.py`
6. `src/perturbdiff/diffusion.py`
7. `src/perturbdiff/losses.py`
8. `tests/test_data.py`
9. `tests/test_model.py`

为什么这样读：

- `train_with_split.py` 告诉你“程序是怎么跑起来的”
- `data.py` 告诉你“训练样本长什么样”
- `config.py` 告诉你“模型有哪些重要超参数”
- `model.py` 告诉你“这些模块怎么被拼起来”
- `modules.py` 、 `diffusion.py` 、 `losses.py` 是更细的零件实现
- `tests` 则像最短的用法示例

替代读法：

- 有经验的工程师常常会从 `modules.py` 或 `diffusion.py` 这种底层模块开始读
- 这种读法适合已经熟悉 PyTorch 的人
- 对初学者来说，往往会在看到很多类和函数时不知道它们“什么时候被调用、解决什么问题”

所以这次文档故意采用“从入口脚本向下走”的方式。

3. 主流程总览：训练时到底发生了什么

最值得先读的文件是 train_with_split.py。

如果把它简化成伪代码，大概是这样：

```
args = parse_args()
dataset = load_dataset(...)
cfg = ModelConfig(...)
model = PerturbDiffusionModel(cfg)

initialize_perturbation_embeddings(...)
build_signature_gene_weights(...)

for epoch in range(...):
    for batch in train_loader:
        out = model.training_losses(...)
        out["loss"].backward()
        optimizer.step()

evaluate on val/test
save checkpoint
```

这段伪代码很重要，因为它就是你之后读每个模块时要不断回来的“主线”。

下面我们顺着这条主线一步步看。

4. 第一步：读取参数与数据

对应代码位置：

- train_with_split.py: `parse_args`
- train_with_split.py: `load_dataset`

4.1 `parse_args` 在做什么

这里用的是 Python 标准库里的 `argparse`。它负责把命令行参数变成 `args.xxx` 这种可以直接访问的配置对象。

比如：

- `--batch-size`
- `--lr`
- `--prediction-type`
- `--reconstruction-loss`
- `--target-strategy`

为什么这样做：

- 研究代码经常要反复做实验
- 把超参数写在命令行里，复现实验会更方便
- 同一个脚本可以通过不同参数组合跑出不同实验

替代方案：

- 可以改成 YAML/JSON 配置文件
- 配置文件在大型项目里更整齐
- 但这个仓库本身比较紧凑， `argparse` 已经足够清晰，而且对初学者更直观

4.2 `load_dataset` 在做什么

这个函数负责选择具体的数据加载方式。当前 `train_with_split.py` 的主路径是 `.h5ad` 数据，因此最终会调用：

- data.py: `load_h5ad_perturbation_dataset`

为什么把数据读取单独封装：

- 训练脚本应该重点表达“训练流程”
- 数据读取细节如果全塞在 `main()` 里，会很快变得难读

替代方案：

- 把所有读取逻辑写在一个大函数里
- 这样短期看代码跳转少，但长期维护和教学都会更困难

5. 第二步：把标签变成模型能用的整数 id

对应代码位置：

- data.py: `is_control_label`
- data.py: `extract_target_label`
- data.py: `encode_perturbation_labels`

这是初学者很容易忽略、但其实非常关键的一步。

5.1 为什么字符串标签不能直接送进模型

原始数据里的 perturbation 标签通常是字符串，例如：

- `non-targeting`
- `TP53_sg1`
- `TP53_sg2`
- `MYC_sg3`

神经网络里的 `Embedding` 层不能直接吃字符串，它需要整数索引，比如：

- control -> `0`
- TP53 -> `1`
- MYC -> `2`

所以 `encode_perturbation_labels()` 的工作就是把标签映射成整数。

5.2 为什么 control 必须固定为 0

当前项目里有一个很重要的约定：**所有 control 条件都映射到 id 0**。

这样做好处很大：

1. 控制组就有了统一参考点
2. embedding 的第 0 行可以固定为全 0
3. 后面做“相对于 control 的偏移”时逻辑会特别清楚

这也是我在源码里补注释的重点之一。

替代方案：

- 你也可以给不同 control 标签分配不同 id
- 但这样会破坏“control 是统一参考原点”这个设计
- 对这个项目来说，统一成 0 明显更合理

5.3 `target_strategy` 为什么重要

这里有两个常见策略：

- `first_token`
- `raw`

如果标签是 `TP53_sg1` 、 `TP53_sg2` ：

- `first_token` 会把它们都映射到 `TP53`
- `raw` 会保留成两个不同的条件

为什么需要这个开关：

- 有时你只关心 target gene 级别的效果
- 有时你又想保留 sgRNA 级别差异

这个设计比“只支持一种标签粒度”更灵活。

替代方案：

- 可以拆成两个完全不同的数据处理流程
- 但那会让代码重复变多
- 当前这种用参数切换的方式更紧凑，也更适合教学

6. 第三步：把数据包装成 PyTorch Dataset

对应代码位置：

- data.py: `PerturbationDataset`
- data.py: `create_dataloader`

6.1 `PerturbationDataset` 的作用

PyTorch 训练通常不会直接把完整矩阵一次性塞给模型，而是通过 `Dataset` 和 `DataLoader` 按 batch 读取。

这里 `PerturbationDataset` 会保存：

- 表达矩阵 `expression`
- `perturbation_ids`
- `target_ids`
- 可选的 `response_proxy`
- 可选的 `gene_names`

6.2 为什么 `__getitem__` 返回的是字典

当前项目里，一个样本返回的是：

```
{
    "x0": ...,
    "perturbation_ids": ...,
    "target_ids": ...,
}
```

为什么不是元组 `(x0, perturbation_ids, target_ids)`？

因为字典更适合研究代码：

- 可读性更强
- 后面新增字段更方便
- 调用方不容易搞错顺序

替代方案：

- 元组更短
- 但一旦字段变多，新手很容易忘掉“第 2 个位置是什么、第 3 个位置是什么”

所以这里牺牲了一点简洁，换来了更高的可读性。

6.3 为什么要把稀疏行转成 dense tensor

在 `_row_to_tensor()` 里，你会看到对 `.toarray()` 的处理。

这是因为：

- `.h5ad` /scipy 稀疏矩阵的一行，常常不是普通 numpy 数组
- 后面的 MLP 需要稳定的 dense `float32` tensor 输入

为什么这样做：

- 模型逻辑应该尽量只面对一种输入形式
- 把“数据格式差异”挡在 dataset 这一层，后面的模型代码会简单很多

替代方案：

- 让模型自己处理稀疏输入
- 但这样会把数据处理复杂度带进模型层，不利于教学和维护

7. 第四步：用 dataclass 组织配置

对应代码位置：

- config.py

这里定义了三个 dataclass：

- `DiffusionConfig`
- `TrainingLossConfig`
- `ModelConfig`

7.1 为什么不是直接传一堆参数

如果没有 dataclass，模型初始化可能长这样：

```
model = PerturbDiffusionModel(
    n_genes=...,
    n_perturbations=...,
    latent_dim=...,
    encoder_hidden_dims=...,
    denoiser_hidden_dims=...,
    ...
)
```

参数一多，就会变得又长又难检查。

dataclass 的好处是：

- 参数有名字
- 默认值集中写在一起
- 传参时结构更清楚

替代方案：

- 用字典也可以
- 但 dataclass 提供了更好的类型提示和字段可读性

对于初学者来说，dataclass 是非常值得学会的一种代码组织方式。

8. 第五步：构建模型本体

对应代码位置：

- model.py: `PerturbDiffusionModel.__init__`

模型里有四个核心组件：

1. `SemanticEncoder`
2. `PerturbationModule`
3. `ConditionalDenoiserMLP`
4. `GaussianDiffusion`

你可以把它们理解成：

- `SemanticEncoder`：把表达压成 latent 表示
- `PerturbationModule`：存储每个 perturbation 对应的 latent 偏移
- `ConditionalDenoiserMLP`：根据 noisy expression + time + 条件 latent 进行去噪/重建
- `GaussianDiffusion`：负责扩散过程本身的数学公式

为什么拆成四个模块：

- 每个模块职责单一
- 更容易测试
- 更容易替换其中一部分

替代方案：

- 全部写进一个大类
- 这样起步快，但一旦想解释“哪部分在做什么”就会非常痛苦

这个仓库虽然不大，但模块划分是比较健康的。

9. 第六步：为什么要先初始化 perturbation embedding

对应代码位置：

- train_with_split.py: `initialize_perturbation_embeddings`
- model.py: `initialize_perturbation_from_latent_shift`
- modules.py: `initialize_from_latent_shift`

这是整个项目里“方法和实现强绑定”的一个关键点。

9.1 它在做什么

代码不是随机初始化每个 perturbation 向量后直接训练，而是先做一件更有生物含义的事情：

1. 用 encoder 把所有训练细胞编码到 latent 空间
2. 求 control 细胞的平均 latent
3. 求每个 perturbation 条件下细胞的平均 latent
4. 两者相减，得到该 perturbation 的初始偏移向量

也就是：

```
delta_g = mean(latent | perturbation=g) - mean(latent | control)
```

9.2 为什么这样做

因为这比纯随机初始化更“有方向感”。

随机初始化的问题是：

- 参数一开始没有任何生物学含义
- 模型需要花更多训练过程才能自己摸索出“control 到 perturbation 的方向”

当前这种做法借鉴了 scGen 风格的 latent arithmetic，优势是：

- 初始化更合理
- 解释性更强
- 训练初期更稳定

替代方案：

- 纯随机初始化
- 单独预训练 perturbation embedding
- 用更复杂的元学习/对比学习方法初始化

但对这个项目来说，当前做法在“简单”和“有解释性”之间是个很不错的平衡。

10. 第七步：训练时一个 batch 如何流过模型

最重要的函数是：

- model.py: `training_losses`

这基本可以看作整个项目训练逻辑的中心。

10.1 先编码表达得到 `z_sem`

对应代码：

- model.py: `encode_condition`
- modules.py: `SemanticEncoder`

`SemanticEncoder` 本质上是一个 MLP。

它把 `x0` 映射成：

```
z_sem = E(x0)
```

为什么需要 encoder：

- 原始表达维度通常很高
- 模型希望有一个更紧凑、更语义化的条件表示

替代方案：

- 直接把原始表达作为条件输入 denoiser
- 这样当然也能做，但会让去噪器承担更多建模压力，且“perturbation = latent shift”这个想法会不够清晰

10.2 再通过 `PerturbationModule` 组织条件信息

对应代码：

- modules.py: `PerturbationModule.forward`

这里有一个初学者很容易疑惑的点：为什么训练时 `z_pert = z_sem`，而不是 `z_sem + delta`？

这是因为：

- 训练数据里的细胞本来就已经处在它自己的 perturbation 条件下
- 也就是说，这个 `z_sem` 已经是“被 perturb 后的表达”编码出来的 latent

如果你这时再额外加一次 `delta`，就会相当于**重复加 perturbation**。

所以当前实现选择：

- `z_pert = z_sem`
- 同时额外计算 `z_basal = z_sem - delta`

为什么还要保留 `z_basal`：

- 它可用于后面的语义对齐损失
- 也方便把 `delta` 解释成“从 basal 到 perturbed 的偏移”

这是当前项目一个非常关键的设计点，我已经在源码相应位置补了注释。

替代方案：

- 有些方法会显式学习“未扰动 basal state”，然后再加 perturbation
- 那样理论上更干净，但通常需要更强的建模假设或额外监督

当前实现更偏务实：先保证训练逻辑不重复加偏移，同时保留解释空间。

10.3 给 `x0` 加噪声，得到 `x_t`

对应代码：

- diffusion.py: `sample_timesteps`
- diffusion.py: `q_sample`

这一步是标准扩散模型套路：

1. 给每个样本随机采一个时间步 `t`
2. 按这个时间步对应的噪声强度，把高斯噪声混进 `x0`
3. 得到 noisy sample `x_t`

为什么训练时要随机采不同时间步：

- 这样一个 denoiser 才能学会在不同噪声水平下工作

为什么当前项目选择线性 beta schedule：

- 简单
- 标准
- 容易解释

替代方案：

- cosine schedule
- learned schedule

- 其他更复杂的噪声日程

这些当然可行，但会增加理解门槛。当前仓库明显优先保留一个容易看懂的 baseline。

10.4 用 denoiser 做条件预测

对应代码：

- modules.py: `ConditionalDenoiserMLP`

去噪器输入三部分：

1. `x_t`：当前 noisy expression
2. `timesteps`：扩散时间步
3. `z_pert`：条件 latent

为什么时间步不能直接只传一个整数：

- 神经网络不擅长直接理解离散整数“15、200、800”的几何关系
- 所以通常会先把 timestep 做成 embedding

这里用的是：

- `SinusoidalTimeEmbedding`

为什么：

- 这是扩散模型和 Transformer 里都很常见、很经典的做法
- 不需要学习太多额外参数

替代方案：

- 可学习时间 embedding
- 更复杂的 Fourier features

但当前实现保持了最常见、最容易理解的版本。

10.5 预测 `eps` 还是预测 `x0`

对应代码：

- model.py: `training_losses`
- diffusion.py: `prediction_to_x0_and_eps`

当前项目支持两种目标：

- `prediction_type="eps"`
- `prediction_type="x0"`

含义分别是：

- `eps`：模型预测噪声
- `x0`：模型直接预测干净表达

为什么两种都支持：

- 扩散文献里两种都很常见
- 不同任务里它们的训练表现可能不同

为什么这个仓库很重视 `x0` 预测：

- 这里最终关心的是表达重建和反事实表达生成
- 直接在表达空间预测 `x0` 会更贴近任务本身

替代方案：

- 只保留一种预测目标，代码会更短
- 但少了方法层面对比和实验灵活性

11. 第八步：损失函数为什么这样设计

对应代码位置：

- losses.py
- model.py: `training_losses`

当前训练目标可以分成三部分：

1. 主重建损失
2. 可选 semantic alignment
3. 可选 direction alignment

11.1 `weighted_mse_loss`

这是最基础的重建损失。

为什么不是简单 `((pred - target) ** 2).mean()`？

因为项目希望支持**某些基因更重要**的情况，所以引入了 gene weights。

代码里还专门做了一个归一化：

- 先按样本计算加权误差和
- 再除以该样本自己的总权重

为什么这样归一化：

- 避免“权重大”只是因为权重总和更大，而不是因为预测更差

替代方案：

- 直接使用普通 MSE
- 对 baseline 来说这完全合理
- 当前项目之所以保留加权版，是因为它更贴近某些生物任务里的重点基因关注方式

11.2 GEARS 风格 loss

对应代码：

- losses.py: `gears_autofocus_direction_loss`

它包含两层思想：

1. 大误差基因应该受到更多关注
2. 相对于 control mean 的变化方向也很重要

为什么设成可选而不是默认唯一方案：

- 这种 loss 更强调“重要错误”和“方向一致性”
- 但它未必总和主评估目标完全一致

也就是说：

- 它可能更有生物解释性
- 但不一定在所有场景里都是最佳优化目标

因此项目把它设计成一个可以切换的选项，而不是写死。

11.3 semantic alignment loss

对应代码：

- model.py: `training_losses`

它约束的是：

```
z_basal = z_sem - delta
```

希望 perturbed cell 去掉 perturbation 偏移后，能更接近 control 的 basal 状态。

为什么这样做：

- 这会让 `delta` 更像“真正的 perturbation 方向”
- 让 latent arithmetic 的解释更稳定

替代方案：

- 完全不加这个正则
- 对只想先跑通 baseline 的实验来说，这也是很合理的选择

11.4 direction alignment loss

对应代码：

- model.py: `training_losses`

它想表达的是：

- 如果两条 sgRNA 对应同一个 target
- 那它们的 perturbation embedding 方向应该接近
- 但幅度不一定完全一样

这点很重要，因为当前项目希望保留 guide-level 差异。

为什么只对齐方向、不强制完全一样：

- 同 target 的不同 sgRNA 可能强弱不同
- 如果直接共享同一个 embedding，就会抹掉这种差异

替代方案：

- 让同 target 的 sgRNA 共享一个完全相同的 embedding
- 这样更简单，但表达能力更弱

当前实现保留了更细粒度的建模空间。

12. 第九步：为什么反事实生成和训练时不完全一样

对应代码位置：

- model.py: `sample_counterfactual`
- modules.py: `counterfactual_latent`
- diffusion.py: `ddim_sample`

这是另一个非常值得初学者重点理解的地方。

12.1 训练时为什么不显式“加 target perturbation”

因为训练输入本身就是观测到的真实 perturbed cell。

12.2 推理/生成时为什么要显式“减 source、加 target”

因为这时我们做的是**反事实问题**：

> 这一个 control cell，如果换成另一个 perturbation，会变成什么样？

所以这时就必须显式构造目标条件：

1. 先编码源细胞
2. 去掉 source perturbation 的影响
3. 加上 target perturbation 的影响

源码里这一步就在：

- `counterfactual_latent()`

这也是为什么我在这一段专门补了注释：它是“方法概念真正落到代码动作”的地方。

替代方案：

- 可以不用扩散，改成一个直接回归网络一次性输出目标表达
- 这样更快、更简单
- 但就失去了当前扩散生成框架和 DDIM 采样这条方法路线

13. 第十步：DDIM 采样循环怎么理解

对应代码位置：

- diffusion.py: `ddim_sample`

对初学者来说，这段循环往往最容易看晕，因为它既有数学符号，又有很多 timestep 变量。

最简单的理解方式是：

```
从随机噪声开始
反复问 denoiser：“当前这一步的干净样本/噪声应该是什么？”
再按 DDIM 公式走到上一个时间步
一直走到 t = 0
```

当前实现里每次循环干三件事：

1. 取当前时间步 `t`
2. 用 denoiser 得到 prediction
3. 把当前状态更新到前一个时间步

为什么 `GaussianDiffusion` 要把 `alphas_cumprod` 这些量存成 buffer：

- 它们是模型状态的一部分
- 但不是需要训练的参数
- `register_buffer` 能保证 `.to(device)` 时一起搬到 GPU/CPU

替代方案：

- 每次 forward 现算
- 或存成普通 tensor 属性

前者浪费计算，后者设备管理更麻烦，所以 buffer 是很标准的写法。

14. 为什么 denoiser 用的是 MLP

对应代码位置：

- modules.py: `ConditionalDenoiserMLP`

很多人第一次看会问：

> 为什么不是 Transformer？为什么不是图神经网络？

答案很直接：**这个项目优先选择一个紧凑、清楚、容易讲解的 baseline。**

MLP 的优点：

- 最容易读懂
- 适合向量输入
- 训练和调试成本低

替代方案：

- Transformer：更强的表达能力，但代码和计算都更重
- GNN：如果想显式利用基因网络先验，会更有生物动机
- 更深的 ResNet：可能带来更强性能，但也更复杂

为什么当前没选它们：

- 这个仓库明显追求的是“方法核心清晰”
- 对教学来说，这比一开始就堆复杂结构更有价值

15. 为什么项目里有很多小函数

例子：

- `parse_hidden_dims`
- `compute_control_expression_mean`
- `_extract`
- `create_dataloader`

初学者有时会觉得：

> 这些函数看起来很短，为什么不直接写在一起？

因为拆小函数有几个实际好处：

1. 避免重复代码
2. 给一段逻辑起一个清晰名字
3. 更容易单独测试
4. 主流程更容易读

替代方案：

- 全写在 `main()` 里

短期看跳转更少，长期看却更难定位问题，也更不利于教学。

16. 测试文件为什么值得读

对应代码位置：

- test_data.py
- test_model.py

初学者常把测试理解成“只有改代码的人才需要看”。

其实在教学上，测试是非常好的简短示例，因为它们会告诉你：

- 输入应该长什么样
- 输出应该满足什么性质
- 哪些行为是项目作者明确希望保证的

比如测试会验证：

- control 是否保留为 id 0
- `training_losses()` 返回的 loss 是否可反传
- `sample_counterfactual()` 输出形状是否正确

阅读建议：

1. 先看一个测试函数名
2. 猜它想验证什么
3. 再去看对应源码
4. 回来确认自己的理解

这是非常适合初学者的读代码练习方式。

17. 这个项目最重要的几个“为什么”

如果你只记住最核心的设计原因，我建议记住下面这些：

17.1 为什么 control 要固定成 0

因为它是整个 perturbation 空间的参考原点。

17.2 为什么 perturbation embedding 要做 latent mean shift 初始化

因为这样比随机初始化更有解释性，也更接近“从 control 到 perturbation 的方向”。

17.3 为什么训练时不用再给 `z_sem` 加一次 `delta`

因为训练输入本来就是已扰动细胞，再加一次会重复计入 perturbation。

17.4 为什么推理时又要显式加 `delta`

因为推理做的是反事实生成，必须主动构造“目标 perturbation 条件”。

17.5 为什么 direction alignment 只对齐方向

因为同 target 的不同 sgRNA 可能方向相似但强度不同，项目希望保留这种差异。

18. 如果你是 Python 初学者，建议这样学这个仓库

一个很实用的学习顺序是：

1. 先读 smoke_train.py
2. 再读 train_with_split.py
3. 遇到不会的函数，就跳到定义处只读那个函数
4. 重点跟踪这些对象：
    - `args`
    - `dataset`
    - `mapping`
    - `cfg`
    - `model`
    - `batch`
    - `out`
5. 最后用测试文件检验自己理解

为什么推荐先看 `smoke_train.py`：

- 它是最短的端到端例子
- 很适合先建立整体感觉

而 `train_with_split.py` 更像真实实验脚本，细节更多、也更接近正式训练。

19. 读代码时可以不断问自己的三个问题

这是我最推荐初学者养成的习惯。

每读到一段代码，都问自己：

1. 现在被处理的对象是什么？
2. 这一步为什么对方法有帮助？
3. 如果换一种做法，会有什么代价？

比如读 `encode_perturbation_labels()` 时：

1. 处理对象是字符串标签
2. 这一步是为了把标签变成 embedding 可用的整数 id
3. 替代方案是保留字符串到更后面再处理，但那会让模型层更复杂

如果你能用这种方式读完整个仓库，就不只是“看懂代码”，而是在学会研究代码的思考方法。

20. 对应关系速查表

- 方法总览： `README.md`
- 主训练入口： `scripts/train_with_split.py`
- 通用训练脚本： `scripts/train.py`
- 数据处理： `src/perturbdiff/data.py`
- 配置定义： `src/perturbdiff/config.py`
- 模型总装： `src/perturbdiff/model.py`
- 子模块实现： `src/perturbdiff/modules.py`
- 扩散过程： `src/perturbdiff/diffusion.py`
- 损失函数： `src/perturbdiff/losses.py`
- 最小运行例子： `scripts/smoke_train.py`
- 行为示例与验证： `tests/test_data.py` 、 `tests/test_model.py`

21. 这次我在源码里额外补了哪些教学注释

主要加在这些最容易卡住的位置：

- `data.py`
  - 为什么 control 要统一映射为 0
  - 为什么 `target_ids` 和 `perturbation_ids` 可能不同

    - 为什么要把稀疏行转成 dense tensor
- `modules.py`
    - latent shift 初始化的含义
    - 训练时为什么不重复加 perturbation
    - 推理时为什么要“remove source, add target”
- `model.py`
    - `training_losses()` 主流程里每一步的意图
    - semantic / direction alignment 的设计动机
    - 推理与训练条件构造的差异
- `diffusion.py`
    - `_extract()` 为什么要 reshape 广播
    - `q_sample()` 为什么代表 forward diffusion
    - `ddim_sample()` 循环每一步在做什么
- `losses.py`
    - weighted MSE 为什么要按样本总权重归一化
- `train_with_split.py`
    - 为什么 embedding 初始化要看整个训练集
    - 为什么 train/val/test loader 分开
    - 为什么 `training_losses()` 返回的不只是一个标量 loss

22. 最后一句总结

这个项目虽然是一个扩散模型仓库，但它真正最值得学习的地方，不只是公式本身，而是它如何把一个方法拆成清晰的代码层次：

- 数据层负责把原始信息整理成模型能接收的形式
- 配置层负责把实验旋钮收拢到一起
- 模型层负责组织“编码器 + perturbation + diffusion”
- 训练脚本负责把整个流程连起来

当你能把这四层关系看明白时，这个项目就不再是一堆文件，而是一条完整的从 method 到 code 的实现链路。